

CH03 Spring AOP 原理和实现—复习要点

1 AOP 思想概述

1.1 为什么需要 AOP?

[p.6–9]

> **OOP 的局限性** (Limitations of OOP) [p.6]:

- OOP 引入封装、继承和多态建立对象层次结构来模拟公共行为
- 当需要在分散的、不具有继承层次的对象中引入公共行为时，OOP 无法避免代码重复
- OOP 允许定义从上到下的关系（纵向继承），但不适合定义从左到右的关系（横向关注）
- 具体场景：大型系统中的日志记录、性能监控、安全认证、事务控制等代码，平均分散在所有对象层次中，与核心业务功能关系不大

> **具体案例** [p.7]: 每个 DAO 类都需要日志功能；未来日志功能需要统一维护

> **传统解决方案** [p.8]: 纵向继承解决耦合和重复问题——但不够灵活

> **传统方法的三大痛点** [p.9]:

- **代码重复**: 每个业务方法都需编写相同的日志/权限代码
- **耦合度高**: 横切逻辑与核心业务紧密耦合，修改影响面大
- **难以维护**: 跨模块实现一致更新困难

> **AOP 的解决思路** [p.9]: 通过切面 (Aspect) 将横切逻辑抽取出来，统一织入，既保持业务代码简洁，又方便横切逻辑的修改和复用

1.2 什么是 AOP?

[p.10]

> **AOP** (Aspect Oriented Programming): **面向切面编程** [p.10]

> 1990 年由 PARC (Xerox Palo Alto Research Center) 研究人员提出，分析 OOP 局限性后提出 [p.10]

> AOP 采取**横向抽取机制**，取代传统的纵向继承体系解决重复性代码问题 [p.10]

> 典型应用场景：日志记录、安全检查、事务管理、性能监视、数据缓存 [p.10]

> AOP 是对 OOP 的一种**完善和补充**，不是替代 [p.10]

2 AOP 核心概念详解

2.1 七大核心概念

[p.11–13]

> **关注点** (Concern) [p.11]: 程序必须达到的某个目标，分为两类（三棱镜法则）:

- **核心关注点** (Core Concerns): 完成核心业务逻辑的关注点
- **横切关注点** (Cross-cutting Concerns): 代码散落在很多个类或方法之中的关注点

> **切面** (Aspect) [p.11]: 对横切关注点的模块化，将散落在各处的横切代码归整在一起。一般包含**切点** (Pointcut) 定义和**通知** (Advice) 方法

> **连接点** (Join Point) [p.13]:

- 程序执行过程中的某个中断点：方法调用 (Method Invocation)、字段访问 (Field Access)、异常抛出 (Throws)
- **Spring AOP 仅支持方法调用作为连接点**
- 连接点是能插入切面的**候选点集合**
- > **切点 (Pointcut)** [p.13]: 从候选连接点中选择的确定的插入切面的连接点, Spring AOP 中用**表达式**选择
- > **通知/建议 (Advice)** [p.13]: 在切点上运行切面的方式, 分为**五种**: 前置、后置返回、异常、最终、环绕
- > **织入 (Weaving)** [p.13]: 将切面应用到原始/目标对象的方式, 分为动态织入和静态织入
- > **引入/混入 (Introduction / Mixin)** [p.12]: 静态横切的核心技术, 向类中引入新成员 (字段、方法) 或修改继承关系

2.2 AOP 术语速查表

术语 (中文)	英文	核心说明
关注点	Concern	程序目标, 分核心关注点和横切关注点 [p.11]
切面	Aspect	横切关注点的模块化封装 = 切点 + 通知 [p.11]
连接点	JoinPoint	候选中断点, Spring 仅支持方法调用 [p.13]
切点	Pointcut	表达式选出的确定织入位置 [p.13]
通知/建议	Advice	切点上执行的动作: 5 种类型 [p.13]
织入	Weaving	切面应用到目标对象的过程 [p.13]
引入/混入	Introduction	静态横切: 向类中引入新方法/字段 [p.12]

2.3 动态横切 vs 静态横切

[p.12]

- > **动态横切 (Dynamic Crosscutting)** [p.12]:
 - 在程序**运行时**, 通过切入点匹配连接点, 执行通知来增强代码
 - 核心技术: 连接点、切入点、通知/建议、切面
 - 通常用于向对象层次中的方法添加安全验证或日志记录
 - 在很多场景中, 动态横切**基本代表了 AOP**
- > **静态横切 (Static Crosscutting)** [p.12]:
 - 在程序**编译时或编译后**, 不改变原始代码, 向类中引入新成员或修改继承关系
 - 核心技术: 引入/混入 (Introduction/Mixin)
 - 一般应用场景中**很少使用静态横切**
 - 只有 **AspectJ** 能实现静态横切 [p.22]

3 AOP 的实现方式

3.1 三种实现原理

[p.16]

- > **编译期/语言扩展 (Compile-time)** [p.16]:
 - 编译器将切面增加到字节码, 需定义新关键字和扩展编译器

- 代表: **AspectJ** 框架
- > 类加载器 (Class Loader) [p.16]:
 - 目标类装载到 JVM 时, 通过特殊类加载器对字节码重新”增强”
- > 运行时/动态代理 (Runtime / Dynamic Proxy) [p.16]:
 - 目标对象和切面都是普通 Java 类, 通过 JVM 动态代理或第三方库实现运行期动态织入
 - 代表: **JDK 动态代理**、**CGLIB** 第三方动态代理、AspectJ 代理

3.2 JDK 动态代理

[p.17–21]

- > 基于 Java 反射机制 (`java.lang.reflect.Proxy + InvocationHandler`) [p.17]
- > 要求目标类必须实现接口 [p.17]
- > 通过代理类的 `invoke` 方法拦截方法调用, 可添加前置/后置逻辑 [p.18–21]
- > 局限: 只能为实现了接口的类创建代理 [p.30]

3.3 AspectJ 框架

[p.22–27]

- > 功能最完整的 AOP 解决方案和实现技术 [p.22]
- > 通过专门的编译器 (`ajc`) 或类加载器在编译期/编译后/加载时织入切面 [p.22]
- > 具有强大的静态织入和动态织入能力 [p.22]
- > 静态横切只能使用 **AspectJ** 实现 [p.22]
- > Introduction/Mixin 功能 [p.24–25]:
 - 向已有类 (如 `Product`) 引入新接口和方法 (如判断待售状态、排序功能)
 - 非 AOP 方式需要修改硬编码或使用适配器模式
 - AspectJ 通过 `introduction/mixin` 实现, 无需修改原类
- > 缺点: 需引入第三方库, 新语法学习成本较高 [p.22]

3.4 三种实现方式对比

方式	时机	代表技术	特点
编译期	编译时	AspectJ (<code>ajc</code>)	功能最完整, 需特殊编译器 [p.16]
类加载器	加载时	特殊 <code>ClassLoader</code>	字节码增强 [p.16]
运行时	运行时	JDK Proxy / CGLIB	最常用, Spring 默认方式 [p.16]

4 Spring AOP

4.1 Spring AOP 简介

[p.29]

- > Spring 支持多种 AOP 实现: JDK 代理、CGLIB 字节码代理、AspectJ [p.29]
- > 业务应用中只涉及动态横切, 不推荐 AspectJ 的静态横切 [p.29]
- > 从 Spring 3.x 起, 建议使用 **AspectJ 注解方式**实现 AOP [p.29]
- > 两种配置方式: XML 配置和注解方式 [p.29]

4.2 Spring AOP 代理模式

[p.30]

- > **JDK 动态代理** [p.30]:
 - 目标 Bean 实现一个或多个接口时, Spring 默认使用 JDK 动态代理

- 基于接口的代理，代理对象和目标对象实现相同接口
- > **CGLIB 字节码代理** [p.30]:
 - 目标类没有实现接口，或设置 `proxyTargetClass=true` 时使用
 - 通过创建目标类的子类实现代理
 - 限制: `final` 类和 `final/private` 方法不能被 CGLIB 代理
- > Spring 7 引入 `@Proxyable` 注解设置全局代理策略 [p.30]

4.3 JDK 代理 vs CGLIB 代理

特性	JDK 动态代理	CGLIB 代理
代理方式	基于接口	基于子类继承
前提条件	必须有接口	不能是 <code>final</code> 类/方法
性能	反射调用	字节码生成，通常更快
Spring 默认	有接口时默认使用	无接口或强制指定时

4.4 切点表达式语法

[p.32]

- > **execution**: 匹配方法执行，最常用的设计器 [p.32]
 - 示例: `execution(* com.example.service.*.*(..))` 匹配指定包下所有方法
- > **within**: 限制切面在某个包或类内生效 [p.32]
 - 示例: `within(com.example.service..*)`
- > **args**: 按参数类型匹配 [p.32]
 - 示例: `args(java.lang.String, ..)`
- > **@annotation** / **@within** / **@target**: 按注解匹配 [p.32]
- > 支持逻辑运算: `&&` (与)、`||` (或)、`!` (非) [p.32]
- > 可用 `@Pointcut` 定义切点并复用 [p.32]

4.5 五种通知类型

[p.33]

注解	类型	执行时机	适用场景
<code>@Before</code>	前置通知	目标方法执行前	参数校验、前置日志
<code>@AfterReturning</code>	后置返回	方法正常返回后	捕获返回值（不可修改返回值）
<code>@AfterThrowing</code>	异常通知	方法抛出异常后	异常日志、告警
<code>@After</code>	最终通知	方法结束时（无论是否异常）	资源清理
<code>@Around</code>	环绕通知	方法调用前后	事务管理、缓存、性能监控

- > **通知优先级** [p.33]: 多个切面作用于同一连接点时，通过 `@Order` 注解（数值越小优先级越高）或实现 `Ordered` 接口控制执行顺序
- > **最佳实践** [p.33]: 能用其他通知解决时，不要滥用环绕通知

5 Spring AOP 实战要点

5.1 自定义切面注解模式

[p.35–38]

- > 直接使用切面自动装配时，控制切面使用范围不够精确 [p.35]
- > 项目推荐做法 [p.36]：将切面定义成自定义注解，精确控制切入位置
- > 范例流程 [p.37–38]：
 1. 定义自定义注解（如 `@PerformanceMonitor`）
 2. 编写切面类，用 `@Around("@annotation(xxx)")` 绑定
 3. 在需要监控的方法上添加该注解即可

5.2 自调用导致 AOP 失效

[p.39]

- > 问题：同一类中方法 A 调用方法 B，B 上的 AOP 注解不生效
- > 根本原因：自调用不经过 Spring 代理对象，直接调用 `this.methodB()`，绕过了代理
- > 解决方案：
 - 通过 `AopContext.currentProxy()` 获取当前代理对象再调用
 - 将被调用的方法抽取到另一个 Bean 中
 - 在配置中开启 `exposeProxy=true`

5.3 常用 AOP 组件/模块

[p.40]

- > 事务管理 (`@Transactional`) [p.40]：基于环绕通知，调用前开启事务，正常返回后提交，抛异常时回滚
- > 缓存 (`@Cacheable`) [p.40]：环绕通知检查缓存命中，命中则跳过方法返回缓存结果，否则执行方法并写入缓存
- > 异步执行 (`@Async`) [p.40]：代理中将方法执行提交到线程池，实现异步调用
- > 重试与限流 (`@Retryable` / `@ConcurrencyLimit`) [p.40]：拦截方法执行重试逻辑或限制并发调用数

6 IoC 与 AOP 架构全景总结

[p.42]

- > 整个 Spring 架构基于 **IoC/Bean 容器** [p.42]
- > 纵向三层：表示层 (Servlet/JSP) → 业务逻辑层 (Service/Business Bean) → 模型层 (DAO) [p.42]
- > 横向 AOP 三大切面 [p.42]：权限/安全管理切面、日志管理切面、事务管理切面
- > 回顾 IoC 核心 [p.2–3]：解耦合（组件构建与使用分离）、实现基础（面向接口编程 + 工厂模式 + 依赖注入）、运行时注入 [p.3]

7 本章小结

[p.41]

- > **AOP 思想** [p.6–10]：OOP 局限性 → AOP 横向抽取机制 → 典型应用场景
- > **七大核心概念** [p.11–13]：Concern / Aspect / JoinPoint / Pointcut / Advice / Weaving / Introduction —— 中英文名称必考
- > **动态横切 vs 静态横切** [p.12]：运行时增强 vs 编译时引入
- > **三种实现原理** [p.16]：编译期 (AspectJ) / 类加载器 / 运行时 (JDK Proxy, CGLIB)
- > **Spring AOP** [p.29–33]：JDK vs CGLIB 代理、切点表达式 (execution/within/args)、五种通知类型、`@Order` 优先级

> 实战要点 [p.35–40]: 自定义切面注解模式、自调用 AOP 失效问题、四大常用 AOP 组件

高频考点（按重要性排序）:

1. AOP 七大概念中英文名称和含义（Concern/Aspect/JoinPoint/Pointcut/Advice/Weaving/Introduction） [p.11–13]
2. 五种通知类型及其注解、执行时机、适用场景 [p.33]
3. JDK 动态代理 vs CGLIB 代理的区别和适用条件 [p.30]
4. 三种 AOP 实现原理及代表技术 [p.16]
5. 切点表达式语法（execution/within/args 及逻辑运算 &&/||/!） [p.32]
6. 自调用导致 AOP 失效的根本原因和三种解决方案 [p.39]
7. 四种常用 AOP 组件（@Transactional/@Cacheable/@Async/@Retryable） [p.40]
8. 动态横切 vs 静态横切的区别 [p.12]